UNIVERSIDAD DE GRANADA

SISTEMAS MULTIDIMENSIONALES

PRÁCTICA 1

Diseño de Sistemas Multidimensionales

De OLTP a Data Warehouse

Asignatura: Sistemas Multidimensionales

Profesor:
José Ángel Díaz García
joseangeldiazg@ugr.es

FACULTAD DE EDUCACIÓN, ECONOMÍA Y TECNOLOGÍA DE CEUTA

ÍNDICE DE CONTENIDOS

- 1. INTRODUCCIÓN Y OBJETIVOS
- 2. CRONOGRAMA Y ENTREGABLES
- 3. HITO 1: ANÁLISIS DEL SISTEMA REAL
- 4. HITO 2: DISEÑO DE BASES DE DATOS OLTP
- 5. HITO 3: DATA WAREHOUSE DIMENSIONAL
- 6. HITO 4: BENCHMARKING Y EVALUACIÓN
- 7. ANEXOS

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Contexto de la práctica

Los sistemas de información modernos generan grandes volúmenes de datos transaccionales que requieren ser transformados en conocimiento estratégico para la toma de decisiones. Los **Data Warehouses (DW)** y los sistemas multidimensionales proporcionan la arquitectura necesaria para consolidar, organizar y analizar datos históricos de múltiples fuentes, permitiendo a los analistas y directivos responder preguntas complejas de negocio mediante operaciones **OLAP (Online Analytical Processing)**.

Esta práctica tiene como propósito que los estudiantes comprendan y apliquen el ciclo completo de diseño e implementación de un Data Warehouse, desde la identificación de fuentes transaccionales hasta la evaluación de rendimiento de consultas analíticas. Se utiliza la **metodología Kimball** como marco de referencia, haciendo énfasis en conceptos clave como **dimensiones, hechos, granularidad, jerarquías, Bus Matrix y Slowly Changing Dimensions (SCD).**

1.2. Objetivos de aprendizaje

Al finalizar la práctica, el estudiante será capaz de:

- Analizar sistemas transaccionales reales e identificar necesidades analíticas.
- Diseñar esquemas OLTP normalizados para diferentes departamentos empresariales.
- Aplicar metodología sistemática para transformar modelos OLTP en modelos dimensionales.
- Identificar y diseñar dimensiones conformadas compartidas entre áreas de negocio.
- Implementar modelos estrella (Star Schema) y copo de nieve (Snowflake).
- Desarrollar procesos ETL sencillos orientados a (Extract, y Load), el componente de trasnsformación así como proceso ETL se verá en más detalle en la próxima práctica.
- Evaluar rendimiento mediante benchmarking de consultas analíticas.

- Documentar decisiones de diseño y justificar la arquitectura dimensional elegida.

1.3. Caso de ejemplo: Northwind

Como ejemplo, a lo largo del desarrollo de la practica yo os proporcionaré ejemplos ilustrativos. Trabajaremos con una empresa ficticia de distribución comercial llamada **Northwind**, que gestiona una operativa compleja de retail que incluye:

- Productos organizados en categorías.
- Clientes corporativos distribuidos geográficamente.
- Empleados con estructura jerárquica de ventas.
- Pedidos con detalle de productos, cantidades y precios.
- Proveedores internacionales.
- Logística de envíos (transportistas, tiempos, costes).

Áreas departamentales OLTP

1. **Ventas**: gestión de pedidos, clientes, empleados de ventas.
2. **Inventario**: productos, categorías, stock, proveedores.
3. **Logística**: envíos, transportistas, seguimiento.
4. **Recursos Humanos**: empleados, territorios, jerarquías.
5. **Finanzas**: análisis de ingresos, costes, márgenes.

Objetivo analítico: construir un DW que permita responder preguntas como:

- ¿Cuál es la evolución de ventas por categoría, región y trimestre?
- ¿Qué empleados generan mayor margen de beneficio?
- ¿Cuáles son los productos con mayor rotación de inventario?
- ¿Qué transportistas son más eficientes en tiempos y costes?

1.4. Metodología de trabajo

Esta práctica se estructura en **4 hitos secuenciales** que se desarrollan a lo largo de 4 semanas. A diferencia de otras prácticas que requieren entregas independientes en cada fase, aquí se utiliza un **enfoque de documento único incremental**:

- **Al finalizar el Hito 1**, crearás el documento inicial con la **Sección 1 y Sección 2** (análisis del DW real y diseño previo).
- **Al finalizar el Hito 2**, ampliarás ese mismo documento añadiendo la **Sección 3** (diseño OLTP).
- **Al finalizar el Hito 3**, añadirás la **Sección 4** (modelo dimensional y ETL básica).

- **Al finalizar el Hito 4**, completarás el documento con la **Sección 5** (benchmark y optimización).

La entrega final será un **único archivo PDF consolidado**(`practica1_APELLIDOS.pdf`) que contiene las secciones, acompañado opcionalmente de un **repositorio GitHub** con todo el código SQL y scripts Python desarrollados. Si no proporcionas el repositorio de GitHub, deberéis añadir ademas un script completo con todo el proceso de creación del sistema OLTP, OLAP, ETL. **Aquellas partes de la práctica que no pueda probar cargando en mi sistema un script sql no serán evaluadas.**

1.5. Herramientas necesarias

- **PostgreSQL** (motor de base de datos).
- **Algún cliente de SQL para conectar con Postgre.**
- **Python 3.8+**
- **Opcional Git/GitHub** (control de versiones).
- **Draw.io** o similar (diagramas ER y dimensionales).

2. CRONOGRAMA Y ENTREGABLES

Hito	Periodo	Contenido del documento acumulado
1. Analisis y toma de contacto	Semanas 1-2	Análisis del sistema real, e inicio del diseño de nuestro sistema. Necesidades analíticas, diagrama OLAP conceptual.
2. OLTP	Semanas 2-3	Todo lo anterior + diseño de 5 esquemas OLTP, scripts DDL/DML, diagramas ER.
3. Data Warehouse	Semanas 3-4	Todo lo anterior + metodología de transformación, modelos Star/Snowflake, scripts de creación + scripts ETL.
4. Benchmark	Semanas 4-5	Documento completo + benchmarking, análisis comparativo, conclusiones.

3. HITO 1: ANÁLISIS DEL SISTEMA REAL y DISEÑO INICIAL

3.1. Objetivos del Hito 1

El primer hito tiene como finalidad que te familiarices con la estructura de un **Data Warehouse en producción**, es decir, un sistema que ya está siendo utilizado en el mundo real para análisis de datos. Esto te permitirá comprender cómo se organizan las dimensiones, los hechos, las jerarquías y las relaciones en un entorno profesional **antes de diseñar tu propio modelo**. Los objetivos son:

- Comprender el contexto empresarial.
- Analizar y comprender un sistema OLAP real.
- Identificar fuentes de datos transaccionales.
- Definir necesidades analíticas (preguntas de negocio).
- Proponer un modelo multidimensional conceptual.
- Diseñar diagrama OLAP inicial (dimensiones y hechos).

3.2. Análisis de sistema real

Los estudiantes deben buscar en la web y seleccionar un sistema multidimensional real y en producción utilizado por alguna organización o empresa. El sistema debe estar enfocado en el análisis de datos y la toma de decisiones. Algunas organizaciones gubernamentales o del sector público publican paneles de datos o portales de transparencia que utilizan sistemas multidimensionales para mostrar datos estadísticos, indicadores de gestión, etc. (ej., portales de datos abiertos como el que vimos en clase de teoría, dashboards de salud pública, portales de indicadores económicos). Una vez seleccionado el sistema, los estudiantes deben realizar un análisis detallado para identificar los elementos clave del modelo multidimensional y cómo se aplican en ese sistema concreto. Datos a obtener del sistema identificando:

-Objetivo del negocio
-Fuentes de datos
-Dimensiones principales (con jerarquías)
-Hechos y métricas clave
-Tecnología utilizada (si es identificable)
-Preguntas de negocio que el sistema puede resolver.

El objetivo es que os familiaricéis con la presentación de este tipo de sistemas, que información ofrecen, como podemos profundizar y filtrar los datos etc. **Una vez identificado y documentado el sistema real vamos a comenzar con el nuestro.**

Imagina que tenemos que como ingeniero de datos te contratan para modelar el data warehouse de una empresa. A lo largo de esta practica yo usaré Northwind una empresa de retail sencilla como ejemplo, ten en cuenta que las tablas elementos y demás información que yo se presentan en esta memoria ofrecen técnicas avanzadas de SQL, vosotros podéis simplificar los atributos, relaciones, indices, restricciones.

3.3. Diseño del sistema OLAP

Paso 1: Descripción del sistema empresarial

Elaborar una descripción narrativa de la empresa que te ha contratado que incluya:

- **Contexto de negocio**: sector, tamaño, estructura organizativa.
- **Procesos principales**: flujos operacionales de cada departamento.
- **Sistemas actuales**: bases de datos transaccionales existentes.
- **Stakeholders**: quién necesita análisis (gerencia, ventas, logística, finanzas).

Ejemplo para Northwind:

Northwind es una empresa de distribución internacional fundada en 1996 que comercializa productos alimentarios y bebidas. Opera con una plantilla de 9 empleados organizados en territorios de ventas (Norte, Sur, Este, Oeste) y atiende a más de 90 clientes corporativos distribuidos en 21 países.

Los pedidos se procesan diariamente mediante un sistema OLTP que registra: cliente, empleado responsable, fecha de pedido y envío, productos solicitados, cantidades, precios unitarios, descuentos y transportista. Actualmente, cada departamento mantiene su propia base de datos, lo que dificulta el análisis integrado.

Paso 2: Identificación de fuentes de datos

Crear una tabla que enumere las **bases de datos transaccionales existentes, t**en encuesta que ahora estamos identificando las posibles fuentes de datos a integrar en caso de que la empresa existiera y las tuviera, nosotros **tendremos que crearlas.**

Departamento	Sistema OLTP	Entidades principales	Volumen aprox.
Ventas	`oltp_ventas`	Orders, OrderDetails, Customers, Employees	830 pedidos/año
Inventario	`oltp_inventario`	Products, Categories, Suppliers, Stock	77 productos activos
Logística	`oltp_logistica`	Shippers, Shipments, DeliveryTracking	830 envíos/año
RRHH	`oltp_rrhh`	Employees, Territories, EmployeeTerritories	9 empleados

Departamento	Sistema OLTP	Entidades principales	Volumen aprox.
Finanzas	`oltp_finanzas`	Invoices, Payments, CostAnalysis	830 facturas/año

Paso 3: Preguntas de negocio

Formular **5 preguntas analíticas clave** que el Data Warehouse debe responder:

1. **Análisis de ventas**: ¿Cuál es la evolución de ingresos por categoría de producto, país del cliente y trimestre durante los últimos 3 años?
2. **Rendimiento de empleados**: ¿Qué empleados generan mayor volumen de ventas y margen de beneficio por territorio?
3. **Análisis de productos**: ¿Cuáles son los 10 productos más rentables considerando margen unitario, volumen de ventas y rotación de inventario?
4. **Eficiencia logística**: ¿Qué transportistas tienen mejores tiempos de entrega y menores costes por envío, segmentado por región de destino?
5. **Análisis de clientes**: ¿Cuáles son los clientes con mayor frecuencia de compra y ticket promedio, y cómo ha evolucionado su comportamiento?

Paso 4: Decisiones estratégicas

Identificar **2 decisiones empresariales** que dependen de estos análisis:

- **Decisión 1 - Optimización de catálogo**: Basándose en el análisis de rentabilidad por producto y rotación de inventario, decidir qué productos descatalogar, cuáles promocionar y en qué mercados introducir nuevas categorías.
- **Decisión 2 - Reestructuración de territorios de venta**: Evaluar el rendimiento de empleados por territorio para reasignar zonas, definir incentivos y establecer objetivos de ventas realistas basados en históricos.

Paso 5: Modelo OLAP conceptual

Partiendo de las preguntas de negocio, vamos a comenzar a pensar como sería el futuro modelo multidimensional. Por ejemplo:

Pregunta de negocio: *"¿Cuál fue el crecimiento de ventas interanual por categoría de producto en cada región?"*

- Métricas necesarias:
  - Ingresos_totales (SUM de importe_neto) - unidades_vendidas (SUM de cantidad) - margen_bruto (SUM de importe_neto - coste_producto)
- Dimensiones necesarias
  - `dim_tiempo` (año, trimestre, mes) → para análisis temporal
  - `dim_producto` (categoría, subcategoría, producto) → para agrupar por categoría

    - `dim_geografia` (región, país, ciudad) → para análisis geográfico
- Tabla de hechos:
    - `fact_ventas` con granularidad

Para vuestras preguntas de negocio diseñar un **diagrama Bus Matrix** (Kimball) que identifique las dimensiones candidatas y las posibles tablas de hechos. En el ejemplo de retail de Northwind, basado en las preguntas de negocio que hemos definido anteriormente sería algo así:

Dimensiones candidatas:

- `Dim_Tiempo` (fecha, año, trimestre, mes, día semana)
- `Dim_Producto` (producto, categoría, proveedor, precio lista)
- `Dim_Cliente` (cliente, contacto, ciudad, país, región)
- `Dim_Empleado` (empleado, cargo, territorio, supervisor)
- `Dim_Transportista` (transportista, teléfono, región cobertura)
- `Dim_Geografia` (region, país, ciudad)
- `Dim_Proveedor` (proveedor, contacto, ciudad, país)

Hechos candidatos:

- `Fact_Ventas` (order_id, product_id, quantity, unit_price, discount, total, cost, margin)
- `Fact_Envios` (shipment_id, order_id, ship_date, required_date, delivery_date, freight_cost)

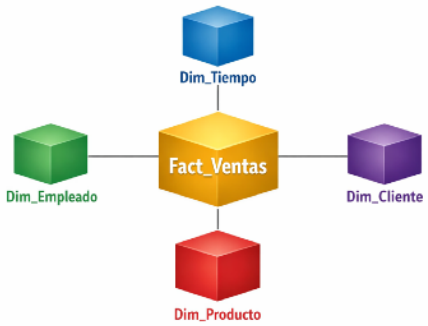
La **Bus Matrix** es una tabla que cruza procesos de negocio (hechos) con dimensiones compartidas. Es fundamental para: Garantizar **consistencia** entre diferentes áreas de análisis , Identificar **dimensiones conformadas** y Planificar el desarrollo incremental del DW.

Bus Matrix para el ejemplo de Northwind:

Proceso de Negocio	Tiempo	Geografia	Producto	Cliente	Empleado	Transportista
Ventas	✓		✓	✓	✓	
Envíos	✓		✓	✓		✓

Paso 6: Diagrama OLAP inicial

Tendréis que añadir los diagramada de diseño OLAP de cada uno de los procesos que necesitéis responder preguntas de negocio. Vosotros tendréis que añadir en vuestros diagramas los principales atributos identificados que tendrá cada tabla de dimension.



3.4. Inicia la documentación

Al finalizar este hito, debéis crear un documento con portada, indice, análisis del sistema real que habéis elegido, definición de la empresa ficticia que os contrata, fuentes de datos OLTP que integrareis, preguntas, decisiones estratégicas, modelado OLAP inicial y gráficos.

4. HITO 2: DISEÑO DE BASES DE DATOS OLTP

En este hito, tendréis que diseñar el modelo OLPT que integrareis en el sistema multidimensional. Tendréis que tener varios departamentos, al menos tendremos que usar 5 departamentos diferentes operacionales. Cada departamento, tendrá al menos 4 tablas que tendremos que poblar con unos 50 registros. Para la creación de datos sintéticos que poblaran vuestros sistemas, utilizaremos la sentencia de DML INSERT, en este caso dado que puede ser un proceso tedioso se recomienda el uso de IA generativa para la creación de las sentencias INSERT.

4.1. Objetivos detallados del Hito 2

- Diseñar 5 esquemas OLTP para los departamentos de Northwind.
- Implementar scripts DDL completos en PostgreSQL.
- Poblar las bases de datos con datos realistas.
- Establecer relaciones entre departamentos (claves foráneas).

4.2. Arquitectura de esquemas OLTP

En el mundo real los sistemas operacionales vivirán en bases de datos distintas. Para simular este entorno, tendremos que crear esquemas distintos que deberemos integrar. De esta manera

aunque no tendremos independencia física real, si que tendremos independencia lógica. Por tanto, cada departamento tendrá su propio esquema PostgreSQL para simular sistemas aislados.

En este punto comenzaremos a proveer ejemplos de codigo SQL que podréis ejecutar en nuestro cliente de Postgre . ten en cuenta que deberas tener una base de datos Postgre creada y corriendo en el servidor local, en el anexo se ofrece información de como hacer esto.

Crearemos los esquemas donde tendremos cada uno de los sistemas operacionales en funcionamiento.

```
CREATE SCHEMA oltp_ventas;
CREATE SCHEMA oltp_inventario;
CREATE SCHEMA oltp_logistica;
CREATE SCHEMA oltp_rrhh;
CREATE SCHEMA oltp_finanzas;
```

4.3. Diseño inicial y sentencias DDL

4.3.1. Diagrama ER

Antes de comenzar con los scripts sql, Tendréis que añadir **un diagrama ER** básico de cada uno de los departamentos. Ten en cuenta que deberas diseñar/modelar 4–6 tablas por departamento (total 20–30 tablas) con:

- Claves primarias
- Claves foráneas y relaciones cruzadas entre departamentos
- Atributos relevantes.
- Constraints (`NOT NULL`, `CHECK`, `UNIQUE`)
- Índices en columnas de consulta frecuente

Una vez diseñado el sistema OLTP, tendremos que crearlo en nuestra pase de datos. Se proporcionan ejemplos para ilustrar lo que se busca en este punto, ten en cuenta que los ejemplos que te proporciono pueden no estar completos, en tu caso **deberán ser ampliados**.

4.3.2. Scripts DDL

```
CREATE TABLE oltp_rrhh.empleados (
empleado_id SERIAL PRIMARY KEY,
nombre VARCHAR(100),
territorio VARCHAR(100)
);


CREATE TABLE oltp_ventas.clientes (
cliente_id SERIAL PRIMARY KEY,
nombre VARCHAR(100),
pais VARCHAR(100),
region VARCHAR(100)
);

CREATE TABLE oltp_ventas.categorias (
```

```
categoria_id SERIAL PRIMARY KEY,
nombre VARCHAR(100)
);

CREATE TABLE oltp_ventas.productos (
producto_id SERIAL PRIMARY KEY,
nombre VARCHAR(100),
categoria_id INT REFERENCES oltp_ventas.categorias(categoria_id),
precio_venta NUMERIC(10,2)
);

CREATE TABLE oltp_ventas.pedidos (
pedido_id SERIAL PRIMARY KEY,
cliente_id INT REFERENCES oltp_ventas.clientes(cliente_id),
empleado_id INT REFERENCES oltp_rrhh.empleados(empleado_id),
fecha_pedido DATE
);

CREATE TABLE oltp_ventas.detalle_pedido (
detalle_id SERIAL PRIMARY KEY,
pedido_id INT REFERENCES oltp_ventas.pedidos(pedido_id),
producto_id INT REFERENCES oltp_ventas.productos(producto_id),
cantidad INT,
precio_unitario NUMERIC(10,2)
);

CREATE TABLE oltp_inventario.costes_producto (
    producto_id INT PRIMARY KEY REFERENCES oltp_ventas.productos(producto_id),
    coste_unitario NUMERIC(10,2)
);

CREATE TABLE oltp_logistica.transportistas (
    transportista_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE oltp_logistica.envios (
    envio_id SERIAL PRIMARY KEY,
    pedido_id INT REFERENCES oltp_ventas.pedidos(pedido_id),
    transportista_id INT REFERENCES
oltp_logistica.transportistas(transportista_id),
    fecha_envio DATE,
    fecha_entrega DATE,
    coste_envio NUMERIC(10,2),
    region_destino VARCHAR(100)
);
```

4.3.3. Scripts DML

Una vez creado todo tendremos que añadir datos sintéticos. Para ello, puedes hacer uso de IA Generativa. Se recomienda poblar las bases de datos OLTP, con datos realistas españoles:

- Nombres y apellidos comunes
- Direcciones reales (calles de ciudades españolas)
- CIF válidos (formato: letra + 8 dígitos + letra)
- Teléfonos (formato: +34-XXX-XXXXXX)
- ≥50 registros por departamento (mínimo 250 registros en total)

```
INSERT INTO oltp_rrhh.empleados (nombre, territorio)
VALUES ('Ana Perez','Sur'),
```

```
('Luis Gomez','Norte');

INSERT INTO oltp_ventas.clientes (nombre,pais,region)
VALUES ('Cliente A','España','Sur'),
('Cliente B','Portugal','Norte');

INSERT INTO oltp_ventas.categorias (nombre)
VALUES ('Bebidas'),('Alimentos');

INSERT INTO oltp_ventas.productos (nombre,categoria_id,precio_venta)
VALUES ('Café',1,10),
('Té',1,8),
('Pan',2,3);

INSERT INTO oltp_inventario.costes_producto
VALUES (1,6),(2,4),(3,1);

INSERT INTO oltp_ventas.pedidos (cliente_id,empleado_id,fecha_pedido)
VALUES (1,1,'2023-02-10'),
(2,2,'2024-05-15');

INSERT INTO oltp_ventas.detalle_pedido
(pedido_id,producto_id,cantidad,precio_unitario)
VALUES (1,1,5,10),
(1,3,10,3),
(2,2,7,8);
```

4.4 Que debéis añadir al finalizar este hito al documento

Al documento que iniciasteis la semana pasada, debéis añadir los diagramas ER de los departamentos, las tablas creadas para cada departamento así como un pequeño ejemplo de las consultas de población (inserts) llevados a cabo. Un posible nombre para esta sección sería: Diseño de fuentes de datos transaccionales y población inicial. Además s**e valorará** la creación de un repositorio en GitHub para ir añadiendo el código de las práctica, el repositorio podría tener esta estructura:

```
/practica1-sm-ugr
  /sql
    /01_ddl_ventas.sql
    /02_dml_ventas.sql
    /03_ddl_clientes.sql
    ...
  /docs
    /diagrama_er_completo.pdf
  README.md
```

5. HITO 3: DATA WAREHOUSE DIMENSIONAL

5.1. Objetivos del Hito 3

En este hito, transformarás tu sistema transaccional (OLTP) en un **modelo dimensional** optimizado para análisis (OLAP). Aprenderás a:

- Transformar los 5 esquemas OLTP en un modelo dimensional integrado.
- Aplicar metodología sistemática para identificar dimensiones y hechos.
- Diseñar dimensiones conformadas compartidas entre procesos de negocio.
- Implementar esquema estrella (Star Schema) optimizado para consultas analíticas.
- Comparar con esquema copo de nieve (Snowflake).
- Desarrollar scripts ETL básicos.

5.2. Metodología y fundamentos de Transformación OLTP → Data Warehouse OLAP

Antes de comenzar cabe recordar las diferencias entre el sistemas OLTP que están en producción en la empresa y los sistemas analíticos que vamos a implementar.

Aspecto	OLTP (Transaccional)	OLAP (Analítico)
Propósito	Operaciones diarias (INSERT/ UPDATE/DELETE)	Análisis histórico (SELECT agregado)
Diseño	Normalización (evitar redundancia)	Desnormalización (optimizar consultas)
Granularidad	Nivel de transacción (cada venta)	Nivel agregado (ventas mensuales)
Volumen	Miles de registros	Millones de registros históricos
Usuarios	Operadores, clientes	Analistas, directivos
Tiempo respuesta	Milisegundos	Segundos/minutos

5.2.1 Mapeo de dimensiones y hechos

Una vez más deberemos comenzar con las preguntas de negocio para identificar que métricas, dimensiones y hechos vamos a necesitar. **Vuelve al paso 5 del hito 1,** para recordar como hicimos esto en nuestro diseño inicial. Para definir si una determinada tabla o item es candidato de ser hecho o dimension podemos usar esta tabla:

Criterio	DIMENSIÓN	HECHO
Volatilidad	Cambia lentamente (cliente, producto)	Cambia constantemente (transacciones)
Cardinalidad	Baja/Media (miles de clientes)	Alta (millones de ventas)
Contenido	Atributos descriptivos (texto, fechas)	Métricas numéricas (importes, counts)
Propósito	Filtrar, agrupar, detallar	Sumar, promediar, calcular ratios
Ejemplo OLTP	Tabla `clientes`, `productos`	Tabla `pedidos`, `detalles_pedid`

Un posible ejemplo de mapeo puede ser:

Tabla OLTP	Tipo	Tabla DW	Justificación
`oltp_ventas.clientes`	**Dimensión**	`dim_cliente`	Atributos descriptivos (nombre, ciudad, segmento), baja volatilidad
`oltp_ventas.productos`	**Dimensión**	`dim_producto`	Jerarquía Categoría → Subcategoría → Producto
`oltp_ventas.empleados`	**Dimensión**	`dim_empleado`	Atributos de vendedor (nombre, departamento, puesto)
`oltp_ventas.detalles_pedido`	**Hecho**	`fact_ventas`	Métricas: cantidad, precio*unitario, descuento, importe*neto
`oltp_inventario.movimientos`	**Hecho**	`fact_inventario`	Métricas: entradas, salidas, stock_final

Para las dimensiones, tendremos también diferentes opciones, es aquí donde entran en juego los Slowly Changing Dimensions (SCD). Tenemos que tener en cuenta que las **dimensiones cambian con el tiempo**. Por ejemplo: - Un cliente se muda de ciudad - Un producto cambia de categoría - Un empleado cambia de departamento. Vamos a tener dientes niveles de dimensiones.

SCD Tipo 0 – Inmutable (Retain Original)

Concepto: El valor **nunca cambia**. Se mantiene el valor original para siempre. **Uso típico:** Atributos que por naturaleza no deben cambiar (ej. fecha de nacimiento, DNI, género al nacer).

SCD Tipo 1 – Sobrescritura (Overwrite)

Concepto: Se **actualiza el valor antiguo** directamente. Se pierde el historial. **Uso típico:** Atributos irrelevantes para análisis histórico (ej. número de teléfono, email de contacto, dirección de envío reciente).

SCD Tipo 2 – Nueva fila con vigencia (Add New Row)

Concepto: Se crea un **nuevo registro** con el valor actualizado, y se mantienen ambas versiones con rangos de fechas de vigencia.Este es el tipo de SCD mas complejo pero al mismo tiempo mas interesante desde el punto de vista analítico. **Uso típico:** Atributos críticos para análisis histórico (ej. ciudad del cliente, categoría del producto, puesto del empleado).

Tipo	Historial	Complejidad	Espacio	Uso típico
0	No	Muy baja	Mínimo	Atributos inmutables (nacimiento, DNI)
1	No	Baja	Mínimo	Atributos irrelevantes (teléfono, email)
2	Completo	Alta	Alto	Análisis histórico (precios, segmentos)

Lo recomendable en el contexto de sistemas multidimensionales, es usar **SCD Tipo 2** para dimensiones clave (cliente, producto, empleado) y **SCD Tipo 1** para atributos secundarios. En está practica no lo tendremos en cuenta, pero es necesario que conozcas como haríamos esto en entornos reales. Además si que tendremos que implementarlo en la práctica 2. Un ejemplo sería, el siguiente. Fíjate que los nuevos atributos de la dimensión, ubicados después del comentario de *control SCD tipo 2*, pueden utilizarse para controlar un histórico de la dimension cliente:

```
CREATE TABLE dim_cliente (
    cliente_sk SERIAL PRIMARY KEY,          -- Surrogate key
(autoincremental)
```

```
    cliente_id INT NOT NULL,                -- antigua key (viene de OLTP)
    nombre VARCHAR(100),
    ciudad VARCHAR(50),
    segmento VARCHAR(20),  -- 'Premium', 'Estandar', 'Ocasional'
    credito_maximo DECIMAL(10,2),

    -- Control SCD Tipo 2
    fecha_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
    fecha_fin DATE DEFAULT '9999-12-31',    -- NULL o fecha lejana =
vigente
    version INT DEFAULT 1,
    activo BOOLEAN DEFAULT TRUE,

    CONSTRAINT uq_cliente_vigente UNIQUE (cliente_id, fecha_fin)
);
```

5.2.2 Mapeo de dimensiones y hechos

Identificar al menos **2 procesos de negocio (hechos)** y marcar qué dimensiones usa cada uno. Tras esto tendréis que crear el bus matrix final del sistema. Como hemos visto anteriormente es una tabla que cruza procesos de negocio (hechos) con dimensiones conformadas.

Proceso (Hecho)	Tiempo	Cliente	Producto	Empleado	Proveedor	Almacén	Canal
Ventas	X	X	X	X		X	X
Inventario	X		X		X	X	
Compras	X		X		X		
Devoluciones	X	X	X	X			X

Las dimensiones conformadas son dimensiones compartidas entre múltiples hechos (ej. `dim_tiempo`, `dim_producto`). Estas nos aportan multiples beneficios como

- **Consultas cruzadas coherentes**: Puedo comparar ventas vs inventario del mismo producto
- **Evitar duplicación de datos**: Un solo `dim_producto` para toda la empresa.
- **Mantenimiento centralizado**: Si actualizo el precio de un producto, se refleja en todos los hechos.
- **Integridad semántica**: Todos los analistas usan las mismas definiciones (ej. "Trimestre Q1 = Enero-Marzo")

5.2.3 Implementamos las dimensiones

```
CREATE SCHEMA olap;
CREATE TABLE olap.dim_tiempo (
tiempo_sk SERIAL PRIMARY KEY,
fecha DATE,
```

```
anio INT,
trimestre INT,
mes INT
);

INSERT INTO olap.dim_tiempo (fecha,anio,trimestre,mes)
SELECT DISTINCT fecha_pedido,
       EXTRACT(YEAR FROM fecha_pedido),
       EXTRACT(QUARTER FROM fecha_pedido),
       EXTRACT(MONTH FROM fecha_pedido)
FROM oltp_ventas.pedidos;

CREATE TABLE olap.dim_cliente (
    cliente_sk SERIAL PRIMARY KEY,
    cliente_id INT,
    nombre VARCHAR(100),
    pais VARCHAR(100),
    region VARCHAR(100)
);

INSERT INTO olap.dim_cliente (cliente_id,nombre,pais,region)
SELECT cliente_id,nombre,pais,region
FROM oltp_ventas.clientes;

CREATE TABLE olap.dim_producto (
    producto_sk SERIAL PRIMARY KEY,
    producto_id INT,
    nombre VARCHAR(100),
    categoria VARCHAR(100),
    precio_venta NUMERIC(10,2),
    coste_unitario NUMERIC(10,2)
);

INSERT INTO olap.dim_producto
(producto_id,nombre,categoria,precio_venta,coste_unitario)
SELECT p.producto_id,p.nombre,c.nombre,
       p.precio_venta,cst.coste_unitario
FROM oltp_ventas.productos p
JOIN oltp_ventas.categorias c ON p.categoria_id=c.categoria_id
JOIN oltp_inventario.costes_producto cst ON p.producto_id=cst.producto_id;

CREATE TABLE olap.dim_empleado (
empleado_sk SERIAL PRIMARY KEY,
empleado_id INT,
nombre VARCHAR(100),
territorio VARCHAR(100)
);

INSERT INTO olap.dim_empleado
SELECT empleado_id,nombre,territorio
FROM oltp_rrhh.empleados;
```

5.2.4 Creamos la tabla de hechos

```
CREATE TABLE olap.fact_ventas (
    venta_sk SERIAL PRIMARY KEY,
    tiempo_sk INT REFERENCES olap.dim_tiempo,
    cliente_sk INT REFERENCES olap.dim_cliente,
    producto_sk INT REFERENCES olap.dim_producto,
```

```
    empleado_sk INT REFERENCES olap.dim_empleado,
    cantidad INT,
    ingresos NUMERIC(12,2),
    coste NUMERIC(12,2),
    margen NUMERIC(12,2)
);
```

5.2.5 ETL básico

El paso final es poblar nuestro sistema dimensional, para ello debemos **extraer l**os datos del sistema OLTP**, transformarlos** para obtener las métricas deseadas (medias, totales, etc…) y **cargarlos** en el DW**.** En la práctica 2, extraeremos los datos y los pasaremos a un area intermedia de stagging, donde aplicaremos técnicas de limpieza avanzadas antes de pasar nuestros datos al DW.

```
INSERT INTO olap.fact_ventas
(tiempo_sk,cliente_sk,producto_sk,empleado_sk,cantidad,ingresos,coste,marge
n)
SELECT
    dt.tiempo_sk,
    dc.cliente_sk,
    dp.producto_sk,
    de.empleado_sk,
    d.cantidad,
    d.cantidad*d.precio_unitario,
    d.cantidad*dp.coste_unitario,
    (d.cantidad*d.precio_unitario)-(d.cantidad*dp.coste_unitario)
FROM oltp_ventas.detalle_pedido d
JOIN oltp_ventas.pedidos p ON d.pedido_id=p.pedido_id
JOIN olap.dim_tiempo dt ON dt.fecha=p.fecha_pedido
JOIN olap.dim_cliente dc ON dc.cliente_id=p.cliente_id
JOIN olap.dim_producto dp ON dp.producto_id=d.producto_id
JOIN olap.dim_empleado de ON de.empleado_id=p.empleado_id;
```

5.2.5 Pasar del la arquitectura en estrella copo de nieve

Para pasar de estrella a copo de nieve debes pasar una de las dimensiones a diferentes tablas de dimension, por ejemplo:

Estrella:

```
dim_producto(
   producto_sk,
   nombre,
   categoria,
   precio_venta,
   coste_unitario
)
```

Se dividiría en dos para hacerlo en copo de nieve:

```
CREATE TABLE olap.dim_categoria (
    categoria_sk SERIAL PRIMARY KEY,
```

```
    nombre VARCHAR(100)
);

CREATE TABLE olap.dim_producto (
    producto_sk SERIAL PRIMARY KEY,
    producto_id INT,
    nombre VARCHAR(100),
    categoria_sk INT REFERENCES olap.dim_categoria(categoria_sk),
    precio_venta NUMERIC(10,2),
    coste_unitario NUMERIC(10,2)
);
```

De esta manera podemos obtener un sistema mas normalizado sin tanta redundancia, aunque solo es aconsejable cuando tenemos un gran número de registros. Las consultas en copo de nieve implican más joins.

5.3. Qué añadir al documento al finalizar este hito

Tendréis que añadir los pasos seguidos para realizar vuestro DW, desde la tabla de mapeo, el bus matrix, los scripts de creación de las dimensiones, los scripts de creación de la tabla de hechos, el script de creación del ETL básico así como explicaciones de las justificaciones que estimes oportunas. Finalmente, tendréis que añadir también el código de como desnormalizariais una dimensión a vuestra elección para pasarlo al modelo estrella.

6. HITO 4: BENCHMARKING Y EVALUACIÓN

En el último hito, evaluarás el rendimiento de tu Data Warehouse mediante la ejecución de **5 consultas analíticas obligatorias**. Tendremos comparar el resultado entre **OLTP vs DW** para las mismas preguntas de negocio, así como la comparativa entre esquemas snow y star. También, aprenderemos como usar python para automatizar este proceso.

6.1. Objetivos del Hito 4

- Diseñar consultas analíticas representativas (OLAP).
- Comparar rendimiento Star Schema vs Snowflake.
- Comparar rendimiento OLTP con OLAP.
- Automatizar benchmarking con Python.

6.2. Consultas analíticas de referencia

Evolución de ingresos por categoría y trimestre:

Star:

```
SELECT
    t.anio,
    t.trimestre,
    p.categoria,
    SUM(f.ingresos) AS total_ingresos
FROM olap.fact_ventas f
JOIN olap.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk
JOIN olap.dim_producto p ON f.producto_sk = p.producto_sk
GROUP BY t.anio, t.trimestre, p.categoria
ORDER BY t.anio, t.trimestre, p.categoria;
```

Snow:

```
SELECT
    t.anio,
    t.trimestre,
    c.nombre AS categoria,
    SUM(f.ingresos) AS total_ingresos
FROM olap.fact_ventas f
JOIN olap.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk
JOIN olap.dim_producto p ON f.producto_sk = p.producto_sk
JOIN olap.dim_categoria c ON p.categoria_sk = c.categoria_sk
GROUP BY t.anio, t.trimestre, c.nombre
ORDER BY t.anio, t.trimestre, c.nombre;
```

Ejemplo de drill down:

Nivel Inicial (Año)

```
SELECT
    t.anio,
    SUM(f.ingresos) AS total
FROM olap.fact_ventas f
JOIN olap.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk
GROUP BY t.anio;
```

Drill Down a Trimestre:

```
SELECT
    t.anio,
    t.trimestre,
    SUM(f.ingresos) AS total
FROM olap.fact_ventas f
JOIN olap.dim_tiempo t ON f.tiempo_sk = t.tiempo_sk
GROUP BY t.anio, t.trimestre;
```

6.3. Script Python de Benchmarking

```
import psycopg2
import time
import statistics

conn = psycopg2.connect(
    host="localhost",
    database="northwind_dw",
    user="postgres",
    password="tu_password"
)

cur = conn.cursor()

# ------------------------------
# CONSULTAS
# ------------------------------

query_oltp = """
SELECT c.nombre, SUM(dv.cantidad * dv.precio_unitario)
FROM oltp_ventas.pedidos p
JOIN oltp_ventas.detalle_pedidos dv
    ON p.pedido_id = dv.pedido_id
JOIN oltp_inventario.productos pr
    ON dv.producto_id = pr.producto_id
JOIN oltp_inventario.categorias c
    ON pr.categoria_id = c.categoria_id
GROUP BY c.nombre;
"""

query_star = """
SELECT p.categoria, SUM(f.ingresos)
FROM olap_star.fact_ventas f
JOIN olap_star.dim_producto p
    ON f.producto_sk = p.producto_sk
GROUP BY p.categoria;
"""

query_snow = """
SELECT c.nombre, SUM(f.ingresos)
FROM olap_snow.fact_ventas f
JOIN olap_snow.dim_producto p
    ON f.producto_sk = p.producto_sk
JOIN olap_snow.dim_categoria c
    ON p.categoria_sk = c.categoria_sk
GROUP BY c.nombre;
"""

# ------------------------------
# FUNCIÓN DE MEDICIÓN
# ------------------------------

def medir(query, repeticiones=20):
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.time()
        cur.execute(query)
        cur.fetchall()
        fin = time.time()
```

```
        tiempos.append(fin - inicio)
    return tiempos

# ------------------------------
# EJECUCIÓN
# ------------------------------

print("Ejecutando OLTP...")
oltp_times = medir(query_oltp)

print("Ejecutando STAR...")
star_times = medir(query_star)

print("Ejecutando SNOW...")
snow_times = medir(query_snow)

# ------------------------------
# RESULTADOS
# ------------------------------

def mostrar(nombre, tiempos):
    print(f"\n--- {nombre} ---")
    print("Promedio:", round(statistics.mean(tiempos), 6), "segundos")
    print("Desv Std:", round(statistics.stdev(tiempos), 6))

mostrar("OLTP", oltp_times)
mostrar("STAR", star_times)
mostrar("SNOW", snow_times)

cur.close()
conn.close()
```

6.4. Plantilla de informe final (Resultados)

Comparación Star vs Snowflake

Deberéis añadir a vuestro pdf una tabla similar a la adjunta con informe del tiempo de cada consulta, deberéis comparar tanto **star** con **snow**, como star con **OLTP**.

Consulta	Star (ms)	Snowflake (ms)	Mejora
Query 1	45.2	67.8	+33%
Query 2	38.6	52.1	+26%
Query 3	62.3	89.5	+30%

6.5 Que debéis añadir al finalizar este hito al documento

Deberéis añadir a vuestro pdf aquellas consultas sql que resuelvan vuestras preguntas de negocio. Como mínimo deberéis añadir una consulta que implique un **roll up** y un **drill down**.

Además de un informe como el que del tiempo que tarda cada consulta tratando de comparar ambos esquemas (STAR vs SNOW vs OLTP). Tendréis que interpretar los resultados y tratar de obtener información sobre cual puede ser el motivo que unas tarden más que otras. Además con el objetivo de que se pueda probar todo vuestro sistemas, deberéis añadir a ala práctica adjunto como ZIP un script sql que pueda ser utilizado para ejecutar toda la creación de esquemas, tablas etc. **Tendréis que añadir también una parte final con las conclusiones del la practica.**

7. ANEXOS

7.1. Anexo A: Guía de instalación de PostgreSQL

Windows:

1. Descargar instalador desde https://www.postgresql.org/download/windows/
2. Ejecutar el instalador (recomendado: PostgreSQL 14 o superior)
3. Configurar (**importante estas serán las directrices para conectar desde python y otras apps externas**):
    - Puerto: `5432` (predeterminado)
    - Usuario: `postgres`
    - Contraseña: [la que elijas]
    - Componentes: PostgreSQL Server + pgAdmin 4 + Stack Builder
4. Verificar instalación: `cmd psql -U postgres`

macOS:

1. Instalar con Homebrew: `bash brew install postgresql@14` `brew services start postgresql@14`
2. Crear usuario: `bash createuser -s postgres`
3. Verificar y conectar: `bash psql -U postgres`

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Configurar usuario
sudo -u postgres psql
ALTER USER postgres WITH PASSWORD 'tu_password';
\q
```

7.2. Anexo B: Comandos útiles de PostgreSQL y plsql

Comando	Descripción
`psql -U postgres`	Conecta con el servidor PostgreSQL.
`create databa\se nombre;`	Para crear la base de datos inicial.
`\l`	Listar todas las bases de datos
`\c nombre_bd`	Conectar a una base de datos
`\dt`	Listar tablas del esquema actual
`\dt dw_northwind.*`	Listar tablas de un esquema específico
`\d+ nombre_tabla`	Describir estructura de una tabla
`\di`	Listar índices
`\dv`	Listar vistas
`\dm`	Listar vistas materializadas
`\timing on`	Activar medición de tiempo de queries
`\x`	Activar modo expandido (vertical)
`\q`	Salir de psql

`\i archivo.sql`	Ejecutar script SQL desde archivo
`\o archivo.txt`	Redirigir salida a archivo

Bibliografía

1. **Kimball, R., & Ross, M. (2013)**. *The Data Warehouse Toolkit*. Wiley.
2. **Abelló, A., & Romero, O. (2020)**. *On-Line Analytical Processing*. Springer.
3. Documentación oficial de PostgreSQL 14.